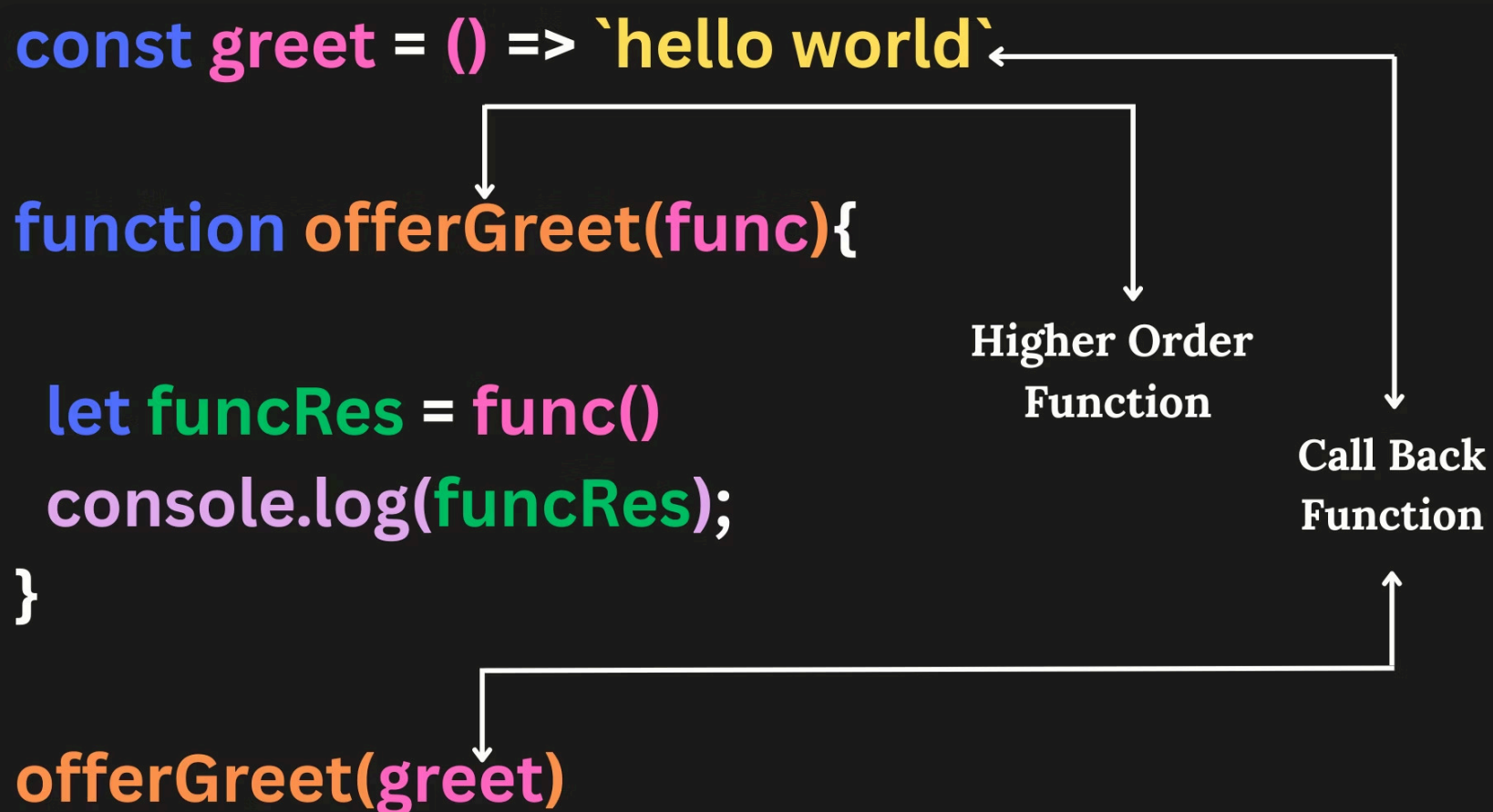


Iterators in JS

Lec-9

Higher Order Functions

- In JavaScript, **functions are first class objects**, this means that like other objects you've encountered JavaScript functions can have properties and methods.
- Functions are special because we can invoke them, but **we can still treat them like any other type of data**.
- A Higher Order Function is a function that **either accepts functions as a parameter, returns a function or does both**.
- The functions which get passed in as parameters are called as - **callback functions**, they **get invoked** during the execution of higher-order function.
- When we **invoke a higher-order function**, and **pass another function as an argument to the callback function** while doing so **we don't invoke the argument to the callback function using open close paranthesis '()'** , as invoking it would also evaluate to pass return value of that function call. Thus we have to **pass in function itself by typing the function name only**.



- **Eg :**

```
const birthYear = (byear) => byear

function currentAge(func, yearVal, cYear = 2026){
  return `Your current age is : ${cYear - func(yearVal)} `
}
let ageRes = currentAge(birthYear, 2001);
console.log(ageRes);
```



Practice Question

Create a function **annualSal** which takes **asalary** as a parameter and returns it.

Create another function **monthlyCredit** which takes 3 parameters- **salaryFunc**, **asal**, **pf** which returns the monthly salary crediting into a person's account.

Inside **monthlyCredit()** pass **asal** as a parameter to the **salaryFunc** and divide it by 12 also subtract 1200 from it as a part of PF contribution.

Display the result by calling **monthlyCredit()** with accurate arguments matching those parameters.

Introduction to Iterators

- Iterators are **built in JavaScript array methods**, that helps us iterate over an array.
- Iterators are **methods** called on **arrays to manipulate elements and return values**.
- All Iterator methods take's **callback function** as an **argument** to **one of their parameters**.

1. .forEach()

- **.forEach()** will execute same code for each element of an array.
- A **callback function** is passed as an **argument** inside **.forEach()**
- **.forEach()** loops through the array and **executes the callback function for each element of that array**.
- During **each execution**, the **current element is passed as an argument to the callback function**.
- The **return value** of **.forEach()** method is always **undefined**.
- **Eg** : Suppose you have an array of some numbers and you wish to print the halves of those numbers consisting in that array, this can be achieved in a following way:

```
const randomNumbers = [80, 40, 20, 10]

randomNumbers.forEach(function iterFunc(numIter){
  console.log(numIter);
})

// This will print 80 40 20 10 to the console.
```

- We can also print indexes and complete array using **.forEach()**

```
const randomNumbers = [80, 40, 20, 10]

randomNumbers.forEach(function iterFunc(numIter,index){
  console.log(`${numIter} is at ${index} index`);
})
/* This will print
  80 is at 0 index
  40 is at 1 index
  20 is at 2 index
  10 is at 3 index */

const randomNumbers = [80, 40, 20, 10]

randomNumbers.forEach(function iterFunc(numIter,index,arr){
  console.log(arr);
})
/* This will print
  [80, 40, 20, 10]
  [80, 40, 20, 10]
  [80, 40, 20, 10]
  [80, 40, 20, 10] */
```

- Suppose we have an array consisting of multiple objects inside it, so by using **.forEach()** we can access each and every property of an array according to our need.

```
const mobDetails = [
  {
    brand: "Apple",
    model: "Iphone",
  },
  {
    brand: "Samsung",
    model: "S25",
  },
  {
    brand: "One Plus",
    model: "Nord CE5",
  }
]
mobDetails.forEach(function(mobItem){
  console.log(mobItem.brand);
  /* This will print :
    Apple
    Samsung
    One Plus */
})
```

Practice Question

You're building a simple movie watchlist feature. You have an array of movies a user has saved to watch later, and you want to display them in the console in a readable format — no filtering, no transforming, just looping through and doing something with each item. Here's your data:

```
const watchlist = [ { title: "Inception", genre: "Sci-Fi", watched: false }, { title: "The Dark Knight", genre: "Action", watched: true }, { title: "Interstellar", genre: "Sci-Fi", watched: false }, { title: "Parasite", genre: "Thriller", watched: true }, { title: "Dune", genre: "Sci-Fi", watched: false } ];
```

Use **.forEach()** to loop through the watchlist and log each movie's title and genre in this format: **"Inception - Sci-Fi"** Modify your **.forEach()** to also print a status alongside each movie — if watched is true, add " Watched", otherwise add " Pending". Example output: **"Inception - Sci-Fi -  Pending"** Bonus: Inside the same **.forEach()**, **keep a count of how many movies are still pending (not watched yet) and log the total count after the loop finishes**

Introduction to Iterators

2. .filter()

- **.filter()** method takes argument as a **callback function**, just like **.forEach()**.
- Similarly like **.forEach()** method **.filter()** also loops through a particular array and returns it's elements.
- The only difference is unlike **.forEach()** method, **.filter()** method **returns the element in new array**.
- Suppose, we have an array and we have to print only those element of an array which are less than 10, by filtering them
- **Eg :**

```
const numArr = [36, 41, 2, 99, 6, 4, 66, 1]
let numArrFilter = numArr.filter((num) => num < 10)
console.log(numArrFilter); // will print [2,6,4,1]
```

```
const mobileApp = [
  {
    appName: 'Whatsapp', launched: 2009, "recent version": "2.26.24.81",
  },
  {
    appName: 'Snapchat', launched: 2011, "recent version": "14.12.0.46",
  },
  {
    appName: 'Instagram', launched: 2010, "recent version": "436.0.0.41",
  },
  {
    appName: 'Facebook', launched: 2004, "recent version": "576.0.0",
  },
  {
    appName: 'Telegram', launched: 2013, "recent version": "12.8.3",
  }
]

let filteringOnLaunch = mobileApp.filter((data) => data.launched < 2010)
console.log(filteringOnLaunch);
/* This will print
{appName: 'Whatsapp', launched: 2009, recent version: '2.26.24.81'}
{appName: 'Facebook', launched: 2004, recent version: '576.0.0'}
*/
```

Practice Question

You're working on a basic task manager for a dev team. The team has a list of tasks logged in the system, and you need to pull out specific subsets of tasks depending on what someone wants to view — that's exactly what **.filter()** is for. Here's your data:

```
const tasks = [ { id: 1, title: "Fix login bug", status: "done", priority: "high" }, { id: 2, title: "Write unit tests", status: "pending", priority: "medium" }, { id: 3, title: "Update README", status: "pending", priority: "low" }, { id: 4, title: "Deploy to staging", status: "done", priority: "high" }, { id: 5, title: "Code review PR #12", status: "pending", priority: "high" }, { id: 6, title: "Fix CSS on mobile", status: "done", priority: "medium" } ];
```

Use **.filter()** to get all tasks where status is "pending" and log the result. Use **.filter()** to get all tasks where priority is "high" and log the result. Chain both conditions together in a single **.filter()** call — tasks that are both "pending" AND "high" priority. These would be the most urgent ones to tackle.

Introduction to Iterators

3. .map()

- **.map()** method takes argument as a **callback function**, just like **.forEach()** and **.filter()**
- **.map()** works in a **similar manner as of .forEach()**, but the only difference is that **.map() returns it's result in a new array**.
- Also **.forEach()** method **doesn't return anything** it's **return value is undefined**, whereas **.map()** method **returns a new array**.
- **Eg:**

```
const numberss = [1, 2, 3, 4, 5, 6, 7, 8, 9]

let numPlusTen = numberss.map((num) => num + 10)

console.log(numberss); // [1, 2, 3, 4, 5, 6, 7, 8, 9]

console.log(numPlusTen); // [11, 12, 13, 14, 15, 16, 17, 18, 19]
```

- Chaining through **.map()** and **.filter()**

```
const fewNums = [1, 2, 3, 4, 5]

let fewNumsChaining = fewNums
    .map((num) => num * 10)
    .map((num) => num + 5)
    .filter((num) => num % 3 == 0)

console.log(fewNumsChaining); // [15,45]
```



Practice Question

You're building a small feature for a developer portfolio site. You have a list of projects stored as raw data objects, and you need to transform them into different shapes depending on what the UI needs to display — that's exactly the job of **.map()**. Here's your data:

```
const projects = [ { title: "weather app", tech: "JavaScript", stars: 12 }, { title: "portfolio site", tech: "HTML/CSS", stars: 34 }, { title: "chat bot", tech: "Python", stars: 8 }, { title: "expense tracker", tech: "React", stars: 21 }, { title: "url shortener", tech: "Node.js", stars: 5 } ];
```

Use **.map()** to return a new array of just the project titles, but with every title converted to Title Case (e.g. "weather app" → "Weather App"). Log the result.

Use **.map()** to return a new array of strings in this format: "weather app — Built with JavaScript". Log the result.

Use **.map()** to return a new array of objects where each object has just two properties — title (in Title Case) and a new property popular which is true if the project has more than 10 stars, and false otherwise.

Introduction to Iterators

4. .reduce()

- The **.reduce()** method **returns a single value after iterating** through the elements of an array, **thereby reducing the array**.
- Syntax :**

```
array.reduce((accumulator, currentValue) => {  
  return accumulator + currentValue;  
});
```

or

```
array.reduce((accumulator, currentValue) => {  
  return accumulator + currentValue;  
}, initialValue);
```

- Eg :**

```
const randNums = [5,7,2,1]  
  
let randomSum = randNums.reduce((accumulator,currentValue) =>{  
  return accumulator + currentValue  
})  
console.log(randomSum); // 15
```

Let's see working of the above code

Iteration	accumulator	currentVal	return
first	5	7	12
second	12	2	14
third	14	1	15

- The **.reduce()** also takes an **optional second parameter** to set an **inital value for accumulator** (first element is callback function)

```
let diffRandomSum = randNums.reduce((accumulator, currentValue) =>{  
  return accumulator + currentValue  
},100)  
console.log(diffRandomSum); // 115
```

Let's see working of the above code

Iteration	accumulator	currentVal	return
first	100	5	105
second	105	7	112
third	112	2	114
fourth	114	1	115

```
const shoppingCart = [  
  {  
    item: "T-Shirts",  
    price: 899  
  },  
  {  
    item: "Watch",  
    price: 599  
  },  
  {  
    item: "Power bank",  
    price: 2200  
  }  
]  
  
let checkOut = shoppingCart.reduce((accumulator,shoppingItems) => accumulator + shoppingItems.price,0)  
console.log(checkOut);
```

 **Practice Question**

You're building the checkout page for a small e-commerce app. You have a list of items in the user's cart, and you need to calculate a few summary numbers to display in the order summary — tallying up values from a whole list into a single result is exactly what .reduce() is built for. Here's your data:

```
const cart = [ { name: "Mechanical Keyboard", price: 80, quantity: 1 }, { name: "USB Hub", price: 25, quantity: 2 }, { name: "Mousepad", price: 15, quantity: 3 }, { name: "Webcam", price: 60, quantity: 1 }, { name: "LED Desk Lamp", price: 20, quantity: 2 } ];
```

Use .reduce() to calculate the total price of the entire cart. Keep in mind each item has a price and a quantity, so the cost per item is price × quantity.

Log the final total. Use .reduce() to calculate the total number of items in the cart (sum of all quantities).

Log the result. Use .reduce() to find the most expensive item (by unit price) in the cart and log just that item's object.

Introduction to Iterators

5. .findIndex()

- **.findIndex()** returns the **index of the first element** that **evaluates to be true in callback function**.
- If there **isn't a single element in the array that satisfies the condition** in the callback, then **.findIndex()** will return **-1**.
- **Eg:**

```
const jumbledNums = [420, 365, 961, 85, 123, 66]

let jumbledOp = jumbledNums.findIndex((num) => num < 100)
console.log(jumbledOp); // 3
```

If no element satisfies the condition:

```
const jumbledNums = [420, 365, 961, 850, 123, 660]

let jumbledOp = jumbledNums.findIndex((num) => num < 100)
console.log(jumbledOp); // -1
```

6. .some()

- **.some()** checks **if at least one element in an array satisfies a given condition**.
- **.some()** method **takes callback function as it's main parameter**.
- The **.some()** returns a **boolean value**:
 1. **true** : if **callback function returns true for at least one element** in the array.
 2. **false** : if **callback function returns false for all elements**.
- The **.some()** method is particularly useful **when you need to check if any element in an array meets a certain condition, without having to iterate through the entire array manually**. It **stops execution as soon as it finds an element that satisfies the condition**, making it efficient for large arrays.

```
const jumbledNums = [420, 365, 961, 85, 123, 66]
let checkNum = jumbledNums.some((num) => num % 2 == 0)
console.log(checkNum);
```

? Practice Question

You're building a simple user management feature for a web app's admin panel. The admin needs to quickly check things about the registered users — like whether a certain type of user exists, or where a specific user sits in the list — without looping through everything manually. Here's your data:

```
const users = [ { id: 101, name: "Alice", role: "admin", active: true }, { id: 102, name: "Bob", role: "editor", active: false }, { id: 103, name: "Charlie", role: "viewer", active: true }, { id: 104, name: "Diana", role: "editor", active: true }, { id: 105, name: "Evan", role: "viewer", active: false } ];
```

Use **.some()** to check if at least one user is currently inactive (**active: false**). Log true or false.

Use **.some()** to check if there is any admin in the list. Log true or false.

Use **.findIndex()** to find the index position of the user named "Diana" in the array. Log the index.

Use **.findIndex()** to find the index of the first inactive user. Log the index.

Use **.findIndex()** to search for a user with id: 999 — someone who doesn't exist — and log a message like "User not found" if the result is -1.